STM32F446RE Bare-Metal Register Guide

Every register the UART / SPI / I2C lab code touches — what it does, which bits we set and why, and exactly where to find it in **RM0390** (the STM32F446xx reference manual).

How to read RM0390

Every register in the code is a base address plus an offset, and RM0390 gives you both: the memory map lists each peripheral's base, and each peripheral chapter ends with a register map of offsets plus one page per register listing every bit.

Peripheral	Base address	Register pages in RM0390 (Rev 9)
RCC	0x40023800	6.3 p.127; register map 6.3.28 p.170
GPIOA / GPIOB / GPIOC	0x40020000 / 0x40020400 / 0x40020800	7.4 p.185; register map 7.4.11 p.191
USART1	0x40011000	25.6 p.814; register map 25.6.8 p.824
SPI2	0x40003800	26.7 p.865; register map 26.7.10 p.874
I2C1	0x40005400	24.6 p.762; register map 24.6.11 p.773

The base addresses come from **2.2.2 Memory map and register boundary addresses, p.57**. Example: `GPIOA_ODR` = 0x40020000 + 0x14 = 0x40020014, which is exactly the number in the code's `#define`.

The code reaches a register with `(*(volatile unsigned int *)0x40020014)`: a pointer to that address, read or written as a 32-bit value. `volatile` tells the compiler the hardware can change it, so every access really happens.

Bit arithmetic used everywhere:

- `(1 << n)` is bit n. Set it with `reg |= (1<<n)`, clear it with `reg &= ~(1<<n)`, test it with `reg & (1<<n)`.

- A 2-bit field for pin n (MODER, PUPDR) sits at bits 2n+1:2n, so `(3 << 2n)` clears it and `(1 << 2n)` or `(2 << 2n)` writes 01 or 10.
- A 4-bit alternate-function field for pin n sits at 4n in AFRL (pins 0 to 7) or 4(n-8) in AFRH (pins 8 to 15).

One trap: chapter 23 (p.681) is **FMPI2C**, a different fast-mode-plus peripheral. The code's I2C1 is chapter **24**, p.744.

RCC: turning the clocks on

A peripheral is dead until its clock-enable bit is set, so this is always the first write; the three registers sit at RCC base 0x40023800 plus 0x30, 0x40 and 0x44.

Register	Address	Bits the code sets	What it powers	RM0390
RCC_AHB1ENR	0x40023830	bit 0 GPIOAEN, bit 1 GPIOBEN, bit 2 GPIOCEN	GPIO ports A (LED, UART pins), B (SPI, I2C pins), C (button)	6.3.10 p.142
RCC_APB1ENR	0x40023840	bit 14 SPI2EN, bit 17 USART2EN, bit 21 I2C1EN	SPI2, USART2 (debug only), I2C1	6.3.13 p.145
RCC_APB2ENR	0x40023844	bit 4 USART1EN	USART1	6.3.14 p.148

Which bus a peripheral hangs on is not obvious: USART1 is on APB2, but USART2, SPI2 and I2C1 are on APB1. The bus is listed per peripheral in the memory map (p.57). Enabling the wrong bus is a classic silent bug: the peripheral's registers read as zero and every `while` wait hangs forever.

The clock itself is the 16 MHz HSI RC oscillator, the reset default (6.2.2 HSI clock, p.121). The code never changes it, so APB1 = APB2 = 16 MHz, and every baud and timing number below is computed from 16,000,000.

GPIO: owning the pins

Four registers configure a pin and two move data; each pin is a 1-, 2- or 4-bit slice of a 32-bit register (7.3.3 I/O port control registers, p.178).

Register	Offset	Bits per pin	How the code uses it	RM0390
GPIOx_MODER	0x00	2	01 = output (LED PA5); 10 = alternate function (UART, SPI, I2C pins); 00 = input (button PC13, the reset default)	7.4.1 p.185
GPIOx_OTYPER	0x04	1	1 = open-drain on I2C PB8/PB9 (I2C lines must be open-drain)	7.4.2 p.185
GPIOx_PUPDR	0x0C	2	01 = pull-up (button PC13, UART RX PA10, I2C SCL/SDA); 10 = pull-down (SPI MISO PB14)	7.4.4 p.186
GPIOx_IDR	0x10	1	read the button: `GPIOC_IDR & (1<<13)`	7.4.5 p.187
GPIOx_ODR	0x14	1	drive the LED: set or clear bit 5 of GPIOA_ODR	7.4.6 p.187
GPIOx_AFRL	0x20	4	pins 0 to 7: PA2/PA3 = AF7 for the USART2 debug port	7.4.9 p.189
GPIOx_AFRH	0x24	4	pins 8 to 15: PA9/PA10 = AF7 USART1; PB13/14/15 = AF5 SPI2; PB8/PB9 = AF4 I2C1	7.4.10 p.189

Why both MODER and AFRH: MODER = 10 hands the pin to some peripheral; AFRH's 4-bit number says which one (7.3.11 Alternate function configuration, p.181). The AF numbers (AF4 = I2C, AF5 = SPI, AF7 = USART) come from the pin multiplexer, 7.3.2 p.176, and the per-pin table in the DS10693 datasheet.

Worked example, PA9 as USART1_TX: MODER bits 19:18 = 10, and AFRH bits 7:4 (pin 9 minus 8 = 1, so 4 x 1) = 7.

Why the pulls matter: an input left floating picks up noise. The UART RX pull-up keeps an unplugged line at idle-high; the SPI MISO pull-down makes an absent slave read as 0x00, which is how the master knows nobody answered.

Button polarity: PC13 reads 1 when released (pull-up) and 0 when pressed, so `pressed = (GPIOC_IDR & (1<<13)) == 0`. Verified on the board.

USART1: the UART link

Four registers run the link; BRR turns the 16 MHz clock into the baud rate, and two flags in SR pace every byte.

Register	Address	Bits used	Purpose	RM0390
USART_SR	0x40011000	bit 7 TXE, bit 5 RXNE	TXE = transmit buffer empty (safe to write DR); RXNE = a byte arrived (read DR)	25.6.1 p.814
USART_DR	0x40011004	8:0	write = send a byte; read = the received byte (and clears RXNE)	25.6.2 p.816
USART_BRR	0x40011008	15:4 mantissa, 3:0 fraction	baud-rate divider	25.6.3 p.817
USART_CR1	0x4001100C	bit 13 UE, bit 3 TE, bit 2 RE	enable the USART, its transmitter, its receiver	25.6.4 p.817

The divider (25.4.4 Fractional baud rate generation, p.787) is USARTDIV = fCK / (16 x baud), stored as a 12-bit mantissa and a 4-bit fraction:

- 9600 baud: 16,000,000 / (16 x 9600) = 104.17, so mantissa 104 = 0x68 and fraction 0.17 x 16 = 3, giving **BRR = 0x0683**.
- 115200 baud (debug port): 16,000,000 / (16 x 115200) = 8.68, so mantissa 8 and fraction 0.68 x 16 = 11 = 0xB, giving **BRR = 0x008B**.

Send and receive follow the manual's procedures (25.4.2 Transmitter p.779, 25.4.3 Receiver p.782): wait for TXE, then write DR; wait for RXNE, then read DR. Pins: PA9 = TX, PA10 = RX, both AF7, wired crossed between the two boards.

UART is symmetric: either board can transmit whenever it likes, which is why the two-way switch demo runs the same code on both boards.

SPI2: the SPI link

Three registers; every SPI transfer is full-duplex, so writing one byte to DR always shifts one byte back in.

Register	Address	Bits used	Purpose	RM0390
SPI_CR1	0x40003800	bit 6 SPE, bit 2 MSTR, bits 5:3 BR, bit 9 SSM, bit 8 SSI, bits 1:0 CPOL/CPHA	enable, master or slave, clock divider, software NSS, mode 0	26.7.1 p.865
SPI_SR	0x40003808	bit 1 TXE, bit 0 RXNE	TX buffer empty; RX byte ready	26.7.3 p.868
SPI_DR	0x4000380C	7:0	write = byte to shift out; read = byte shifted in	26.7.4 p.869

Master CR1 = MSTR + BR 101 + SSM + SSI + SPE: master, clock fPCLK/64 = 250 kHz, software chip-select, enabled. Slave CR1 = SSM + SPE with SSI = 0: slave, permanently selected.

Why SPI2 and not SPI1: SPI1's clock pin SCK is PA5, the same pin as the green LED. Driving a clock there makes the LED unusable, so the code uses SPI2 on PB13 (SCK), PB14 (MISO), PB15 (MOSI), all AF5, and PA5 stays the LED.

Software NSS (26.3.5 Slave select pin management, p.832): with SSM = 1 the chip-select comes from the SSI bit, not from a wire. SSI = 1 keeps the master's internal NSS high so it stays master; SSI = 0 keeps the slave selected. Three wires plus ground, no CS line.

Transfer (26.3.9 Data transmission and reception procedures, p.837): wait TXE, write DR, wait RXNE, read DR; the byte read is the one the slave had loaded. The slave answers by writing its reply into DR ahead of time, so the master sees it on the next transfer, one step late. Mode 0, CPOL = CPHA = 0 (26.3.6 Communication formats, p.834).

SPI is master-driven: only the master generates SCK (26.3.1 General description, p.827), so the slave cannot start a transfer. For slave-to-master data such as the slave's button, the master polls, reading the slave's byte every 10 ms.

I2C1: the I2C link

Eight registers; CR2, CCR and TRISE set 100 kHz from 16 MHz, and the SR1 flags gate each step of the start, address, data, stop sequence.

Register	Address	Bits used	Purpose	RM0390
I2C_CR1	0x40005400	bit 0 PE, bit 8 START, bit 9 STOP, bit 10 ACK, bit 15 SWRST	enable; generate start and stop; send ACK; software reset	24.6.1 p.762
I2C_CR2	0x40005404	5:0 FREQ = 16	the peripheral clock in MHz	24.6.2 p.764
I2C_OAR1	0x40005408	7:1 address, bit 14 = 1	the slave's own address 0x28; bit 14 must be kept at 1	24.6.3 p.765
I2C_DR	0x40005410	7:0	the address plus R/W byte, then data	24.6.5 p.766
I2C_SR1	0x40005414	bit 0 SB, bit 1 ADDR, bit 2 BTF, bit 4 STOPF, bit 6 RXNE, bit 7 TXE, bit 10 AF	start sent; address acked; byte done; stop seen; data in or out; **AF = no acknowledge**	24.6.6 p.766
I2C_SR2	0x40005418	bit 1 BUSY, bit 2 TRA	bus busy; on the slave, TRA = 1 means the master is reading from me	24.6.7 p.769
I2C_CCR	0x4000541C	11:0 = 80	SCL speed	24.6.8 p.770
I2C_TRISE	0x40005420	5:0 = 17	maximum SCL rise time	24.6.9 p.771

Timing from the 16 MHz clock, where one cycle is 62.5 ns: CCR = fPCLK / (2 x fSCL) = 16,000,000 / (2 x 100,000) = 80, and TRISE = (1000 ns / 62.5 ns) + 1 = 17.

Master write (24.3.3 I2C controller mode, p.749): set START, wait SB, write DR = (addr << 1) + 0, wait ADDR (or AF if nobody answered), read SR1 then SR2 (that pair clears ADDR, 24.6.6), wait TXE, write DR = data, wait BTF, set STOP.

Master read of one byte: ACK = 1, START, wait SB, DR = (addr << 1) + 1, wait ADDR, ACK = 0 so the single byte is NACKed, clear ADDR, set STOP, wait RXNE, read DR.

Slave (24.3.2 I2C target mode, p.747): wait ADDR, read SR2 and check TRA. TRA = 0 means receive: wait RXNE, read DR, then STOPF. TRA = 1 means transmit: wait TXE, write DR, then AF when the master NACKs.

Two hardware rules: SCL and SDA are open-drain and need pull-ups (24.3.6 SDA/SCL line control, p.755), and bit 10 AF is the "no slave" detector. The code's guard counters make a missing slave return 0 instead of hanging.

Line by line: every register write in the code

Each row is one statement from the lab code, the register it touches, the bits, and why.

Code (what it writes)	Register	Bits	Why
`RCC_AHB1ENR` set bits 0, 1, 2	RCC_AHB1ENR	GPIOAEN, GPIOBEN, GPIOCEN	clock to the ports the LED, bus pins and button live on
`RCC_APB2ENR` set bit 4	RCC_APB2ENR	USART1EN	clock to USART1
`RCC_APB1ENR` set bit 14	RCC_APB1ENR	SPI2EN	clock to SPI2
`RCC_APB1ENR` set bit 21	RCC_APB1ENR	I2C1EN	clock to I2C1
`GPIOA_MODER` clear (3<<10), set (1<<10)	GPIOA_MODER	11:10 = 01	PA5 is an output: the LED
`GPIOA_MODER` set (2<<18), (2<<20)	GPIOA_MODER	19:18, 21:20 = 10	PA9, PA10 belong to a peripheral
`GPIOA_AFRH` set (7<<4), (7<<8)	GPIOA_AFRH	7:4, 11:8 = 7	that peripheral is USART1 (AF7)
`GPIOA_PUPDR` set (1<<20)	GPIOA_PUPDR	21:20 = 01	pull-up on PA10 RX: no noise when unplugged
`GPIOC_PUPDR` set (1<<26)	GPIOC_PUPDR	27:26 = 01	pull-up on PC13: button reads 1 until pressed
`(GPIOC_IDR & (1<<13)) == 0`	GPIOC_IDR	bit 13	button pressed (active-low)

Code (what it writes)	Register	Bits	Why
`GPIOA_ODR` set or clear (1<<5)	GPIOA_ODR	bit 5	LED on or off
`USART1_BRR = 0x0683`	USART_BRR	15:0	9600 baud at 16 MHz
`USART1_CR1 = UE + TE + RE`	USART_CR1	13, 3, 2	enable USART, transmitter, receiver
`while (!(USART1_SR & (1<<7)))`	USART_SR	bit 7 TXE	wait until the transmit buffer is free
`USART1_DR = 'S'`	USART_DR	7:0	send the byte
`while (!(USART1_SR & (1<<5)))`	USART_SR	bit 5 RXNE	wait for a byte to arrive
`GPIOB_AFRH` set (5<<20), (5<<24), (5<<28)	GPIOB_AFRH	23:20, 27:24, 31:28 = 5	PB13/14/15 are SPI2 (AF5)
`GPIOB_PUPDR` set (2<<28)	GPIOB_PUPDR	29:28 = 10	pull-down on PB14 MISO: an absent slave reads 0x00
`SPI2_CR1 = MSTR + BR 101 + SSM + SSI + SPE`	SPI_CR1	2, 5:3, 9, 8, 6	master, fPCLK/64, software NSS high, enabled
`SPI2_CR1 = SSM + SPE`	SPI_CR1	9, 6	slave, software NSS low (selected), enabled
`while (!(SPI2_SR & (1<<1)))` then `SPI2_DR = b`	SPI_SR, SPI_DR	TXE, 7:0	shift a byte out
`while (!(SPI2_SR & (1<<0)))` then read `SPI2_DR`	SPI_SR, SPI_DR	RXNE, 7:0	take the byte that shifted in
`GPIOB_OTYPER` set bits 8, 9	GPIOB_OTYPER	8, 9	PB8/PB9 open-drain, as I2C requires
`GPIOB_AFRH` set (4<<0), (4<<4)	GPIOB_AFRH	3:0, 7:4 = 4	PB8/PB9 are I2C1 (AF4)

Code (what it writes)	Register	Bits	Why
`I2C1_CR1` set then clear bit 15	I2C_CR1	SWRST	software reset to a clean state
`I2C1_CR2 = 16`, `CCR = 80`, `TRISE = 17`	CR2, CCR, TRISE	5:0, 11:0, 5:0	16 MHz in, 100 kHz SCL, 1000 ns rise
`I2C1_OAR1 = (0x28<<1) + (1<<14)`	I2C_OAR1	7:1, bit 14	slave address 0x28; bit 14 must be 1
`I2C1_CR1` set bit 8	I2C_CR1	START	start condition
`while (!(I2C1_SR1 & (1<<0)))`	I2C_SR1	bit 0 SB	start condition went out
`I2C1_DR = (addr<<1) + 0`	I2C_DR	7:0	slave address plus write bit
wait `SR1` bit 1 or bit 10	I2C_SR1	ADDR / AF	acked, or NACK: no slave on the bus
`(void)I2C1_SR1; (void)I2C1_SR2;`	SR1, SR2	none	reading both clears ADDR (24.6.6)
`I2C1_CR1` set bit 9	I2C_CR1	STOP	stop condition
`I2C1_SR2 & (1<<2)`	I2C_SR2	bit 2 TRA	slave: 1 = the master is reading from me

Explaining it to the teacher

Walk the manual in the same order the code runs, pointing at one page per step.

1. **Addresses** (p.57): show that every `#define` is a base from the memory map plus the offset from that chapter's register map. Example: GPIOA_ODR = 0x40020000 + 0x14.
2. **Clock first** (p.142, 145, 148): nothing works until RCC's enable bit is set; name the bus (AHB1 for GPIO, APB2 for USART1, APB1 for SPI2 and I2C1).
3. **Pins** (p.185 to 189): MODER chooses output or alternate function, AFRH chooses which peripheral (AF7 USART, AF5 SPI, AF4 I2C), PUPDR and OTYPER set the electrical behaviour.

4. **The peripheral's own procedure**: open the manual's step list and show the code mirrors it: USART transmitter and receiver (p.779, 782), SPI transmission procedure (p.837), I2C controller and target mode (p.749, 747). The pattern is always wait for a status flag, then touch DR.
5. **The numbers**: derive BRR 0x0683 from 16 MHz and 9600 baud, and CCR 80 and TRISE 17 from 16 MHz and 100 kHz. Doing the arithmetic on the board is worth more than reciting it.
6. **Design choices**: SPI2 instead of SPI1 (SCK would steal the LED's PA5), PC13 active-low with a pull-up, open-drain I2C with pull-ups, and the no-link-no-blink rule (I2C AF bit, SPI read-back, UART timeout).

Questions to expect, and the one-line answers:

- *Why no HAL?* Every line maps to a page in RM0390; it shows the hardware is understood, not hidden behind a library.
- *Why read SR1 then SR2 after ADDR?* 24.6.6 (p.766): ADDR is cleared by reading SR1 followed by SR2; skip it and the transfer stalls.
- *How does the master know the slave is missing?* I2C: no ACK sets AF. SPI: the read-back byte is not the ack value. UART: the reply never arrives before the timeout.
- *Why is SPI master-driven?* Only the master generates SCK (26.3.1, p.827); the slave can only answer, so the master polls for the slave's button.
- *Why 16 MHz?* The HSI oscillator is the reset default (p.121) and the code never switches to the PLL, so every divider uses 16,000,000.

The code: what every file shares

Every program has the same skeleton: enable clocks, configure pins, configure the peripheral, then loop forever waiting on status flags. Nothing uses interrupts; the CPU polls.

The pieces that appear in all six files:

Function or line	What it does	Why it is there
`SystemInit()`, `__libc_init_array()`	empty stubs, marked `weak`	the startup file calls both before `main`; empty keeps the 16 MHz reset clock, and `weak` lets CubeIDE's own versions override them

Function or line	What it does	Why it is there
`delay_ms(ms)`	spins ms x 1300 empty loops	about 1 ms per unit at 16 MHz; approximate, no timer needed
`pressed()`	returns 1 when PC13 reads 0	the USER button pulls the pin low, so low means pressed
`blink(n)`	LED on 200 ms, off 200 ms, n times	"the other board asked me to blink" (5 times)
`little_blink()`	LED on 120 ms, off 120 ms, twice	"my request was acknowledged"
`now && !prev`	true only on the first loop pass where the button is down	one press gives one action, even if the button is held
`delay_ms(20); if (pressed())`	re-check 20 ms later	debounce: a mechanical button bounces for a few ms
`while (!(REG & FLAG));`	spin until the hardware sets a flag	the polling pattern behind every send and receive

The `main()` order is always the same, and it matters:

1. RCC: set the clock-enable bit for every GPIO port and peripheral used.
2. GPIO: MODER (mode), AFRH (which peripheral), PUPDR and OTYPER (electrical).
3. Peripheral: baud or speed registers, then the enable bit last.
4. `while (1)`: the protocol loop.

A pin configured before its port clock is on silently keeps its reset value, so step 1 must come first.

UART two-way program, step by step

Both boards run the same file: each has a button and an LED, and either one can start the exchange because UART lets both sides transmit at any time.

The two bytes on the wire:

Byte	Sent by	Meaning	Receiver's reaction
'S'	the board whose button was pressed	"your turn, blink"	`blink(5)`, then reply 'D'
'D'	the board that just blinked	"done, acknowledged"	`little_blink()` if it was waiting

One pass of the loop, every millisecond:

1. Read the button. On a fresh, debounced press: send 'S', set `awaiting = 1` and `timer = ACK_TIMEOUT` (6000).
2. If a byte is waiting (RXNE): 'S' means the other board pressed, so `blink(5)` and send 'D'; 'D' while `awaiting` means my press was acknowledged, so `little_blink()` and clear `awaiting`.
3. If still `awaiting`, count `timer` down; at zero give up quietly, because the other board is not connected.
4. `delay_ms(1)` and repeat.

Why the numbers: `blink(5)` blocks for about 2 s, so the timeout must be longer than that or a real reply would be missed; 6000 ms gives a safe margin. `send()` waits for TXE before writing DR, so a byte is never overwritten mid-transmission.

What you see: press either button, the other board blinks 5 times, then the pressed board gives 2 short blinks. Nothing pressed, both boards stay dark.

SPI master and slave programs

SPI has one clock master, so the master's loop drives both directions: it sends its own button press, and it polls to discover the slave's.

Bytes on the bus (the master sends on MOSI, the slave answers on MISO in the same transfer):

Byte	Direction	Meaning
'S'	master to slave	master's button pressed: slave, blink 5
0x00	master to slave	idle poll: slave, report your button
'K'	master to slave	I saw your press and blinked: slave, little-blink

Byte	Direction	Meaning
'B' or 0x00	slave to master	my button is pressed, or not

`spi_xfer(b)`: wait TXE, write b to DR, wait RXNE, return DR. One call both sends a byte and returns the byte the slave shifted back.

Master loop, every 10 ms:

1. Fresh press: `spi_xfer('S')`, then `delay_ms(2500)` so the slave can finish its 5 blinks, then `little_blink()`.
2. Poll: `r = spi_xfer(0x00)` and `nowB = (r == 'B')`. If `nowB && !prevB`, a fresh press on the slave: `blink(5)`, then `spi_xfer('K')`. Save `prevB = nowB`.

Slave loop:

1. Preload DR = 0x00 before the first transfer.
2. Wait RXNE and read the command: 'S' means `blink(5)`; 'K' means `little_blink()`; anything else is a poll.
3. Wait TXE and load DR = 'B' if the button is pressed, else 0x00, ready for the master's next transfer.

Three details that make it work: the slave's reply is read one transfer late, because a byte loaded into DR only leaves on the next clock; `prevB` edge detection means a held button blinks the master once, not continuously; and the MISO pull-down makes a missing slave read 0x00, never 'B'.

I2C master and slave programs

I2C is also master-driven, so the master writes commands to the slave and reads the slave's button back; every bus step waits on an SR1 flag, with a loop guard so a missing slave can never hang the program.

`i2c_write(addr, data)` returns 1 only if the slave acknowledged:

1. Wait until BUSY clears.
2. Set START, wait SB.
3. DR = (addr << 1) + 0, the address with the write bit.
4. Wait for ADDR or AF. AF means no acknowledge, no slave: clear it, set STOP, return 0.
5. Read SR1 then SR2 to clear ADDR.

6. Wait TXE, DR = data, wait BTF, set STOP, return 1.

`i2c_read(addr, &out)` fetches one byte:

1. Set ACK, wait BUSY clear, START, wait SB.
2. DR = (addr << 1) + 1, the read bit. Wait ADDR or AF (AF: no slave, return 0).
3. Clear ACK so the single byte is NACKed, clear ADDR, set STOP.
4. Wait RXNE, read DR into `out`, return 1.

Every wait is `while (!(flag) && --g)` with `g = GUARD` (200,000): if the flag never comes, the loop ends and the function returns 0 instead of freezing.

Master loop, every 10 ms:

1. Fresh press: if `i2c_write(0x28, 'S')` returned 1, `delay_ms(2500)` while the slave blinks, then `little_blink()`.
2. Poll: `i2c_read(0x28, &r)`; a fresh `r == 'B'` (edge on `prevB`) means `blink(5)`, then `i2c_write(0x28, 'K')`.

Slave loop:

1. Wait ADDR (the master used my address), then read SR2 and look at TRA.
2. TRA = 0, the master is writing: wait RXNE, read the command, wait STOPF and clear it. 'S' means `blink(5)`, 'K' means `little_blink()`.
3. TRA = 1, the master is reading: wait TXE, DR = 'B' or 0x00 from the button, then wait AF (the master's NACK ends the byte) and clear it.

Why these rules exist: the SR1-then-SR2 read is how the hardware clears ADDR (RM0390 24.6.6); NACKing the last byte tells the slave to stop driving SDA; and OAR1 bit 14 must be 1 or the slave never matches its address.